



National University of Computer & Emerging Sciences, Karachi

Fall-2025 School of Computing (BSCS, BSSE, BSCY, BSAI)

Assignment # 2 Solution

Subject: Database Systems -CS2005
Total Marks: 50

Post Date: 8/10/2025
Due Date: 24/10/2025

Course Instructors: Dr. Zulfiqar, Dr. Farrukh Salim, Basit Jasani, Ali Naseer, Atiya Jokhio, Javeria Farooq, Hajira Ahmed, Huzaifa, Umair

Instructions to be strictly followed.

- For all questions involving SQL Queries:
 - o **Submit the SQL Scripts in a .txt file.**
- It should be obvious that submitting your work after the due date will result in zero points being awarded.
- Plagiarism (copying/cheating) and late submissions result in a zero mark.

Question #1: A film studio needs a database to track its **Films**, the **Actors** they hire, and the **Production Companies** they work with.

Business Rules:

1. **Films:** Every **Film** has a unique FilmID, a Title, a Budget, and a ReleaseDate.
2. **Actors:** Every **Actor** has a unique ActorID, a Name, and a Phone number.
3. **Casting:** A **Film** casts many **Actors**, and an **Actor** can be cast in many **Films**. The role an actor plays (CharacterName) must be recorded when they are cast in a film.
4. **Production:** Each **Film** is made by exactly one **Director** (who is not tracked separately as an entity). The director's Name is stored directly in the FILM record.
5. **Funding:** A **Film** is funded by one or more **Production Companies** (CompanyID, Name). A company can fund many films.

- A. Identify all primary Entities and their corresponding Attributes from the scenario. Underline the Primary Key (PK) for each entity.
- B. Identify the three main Relationships (e.g., A relates to B). State the Cardinality (1:1, 1:M, M:N) for each relationship.
- C. Rule 3 describes a Many-to-Many (M:N) relationship. Name the Associative Entity (Linking Table) needed to resolve it and list all the attributes it will contain (including the PK).
- D. Explain why DirectorName (Rule 4) should NOT be made into a separate entity in this simplified scenario.

Solution:

A.

Entities and Attributes with Primary Keys (PK):

1. FILM (FilmID, Title, Budget, ReleaseDate, DirectorName)
2. ACTOR (ActorID, Name, Phone)
3. PRODUCTION_COMPANY (CompanyID, Name, Address)

B.

Relationships and Cardinality:

1. FILM casts ACTOR: M:N (Many films cast many actors).
2. FILM is funded by PRODUCTION_COMPANY: M:N (Many films funded by many companies).
3. FILM has DirectorName: 1:1 (Implied, as DirectorName is an attribute of Film).

C.

Associative Entity (Linking Table): CASTING

This entity links FILM and ACTOR and holds the CharacterName attribute.

Attributes: FilmIDFK, ActorIDFK, CharacterName

Composite Primary Key (PK): FilmID, ActorID

D.

DirectorName should not be made into a separate entity because the business rule states that we do not track any other information about the director (like their ID, specialization, or phone number). Since a DirectorName is dependent only on the FilmID and provides no other independent data, treating it as a simple attribute of FILM is the easiest design choice.

Question #2: Design an ER Schema for keeping track of information about an Airline Reservation System, given the description below:

There are 6 different airlines in 6 different countries: Canada – AirCan, USA - USAir, UK - BritAir, France

- AirFrance, Germany - LuftAir, Italy - ItalAir. Their flights involve the following 12 cities: Toronto and Montreal in Canada, New York and Chicago in US, London and Edinburgh in UK, Paris and Nice in France, Bonn and Berlin in Germany, Rome and Naples in Italy.

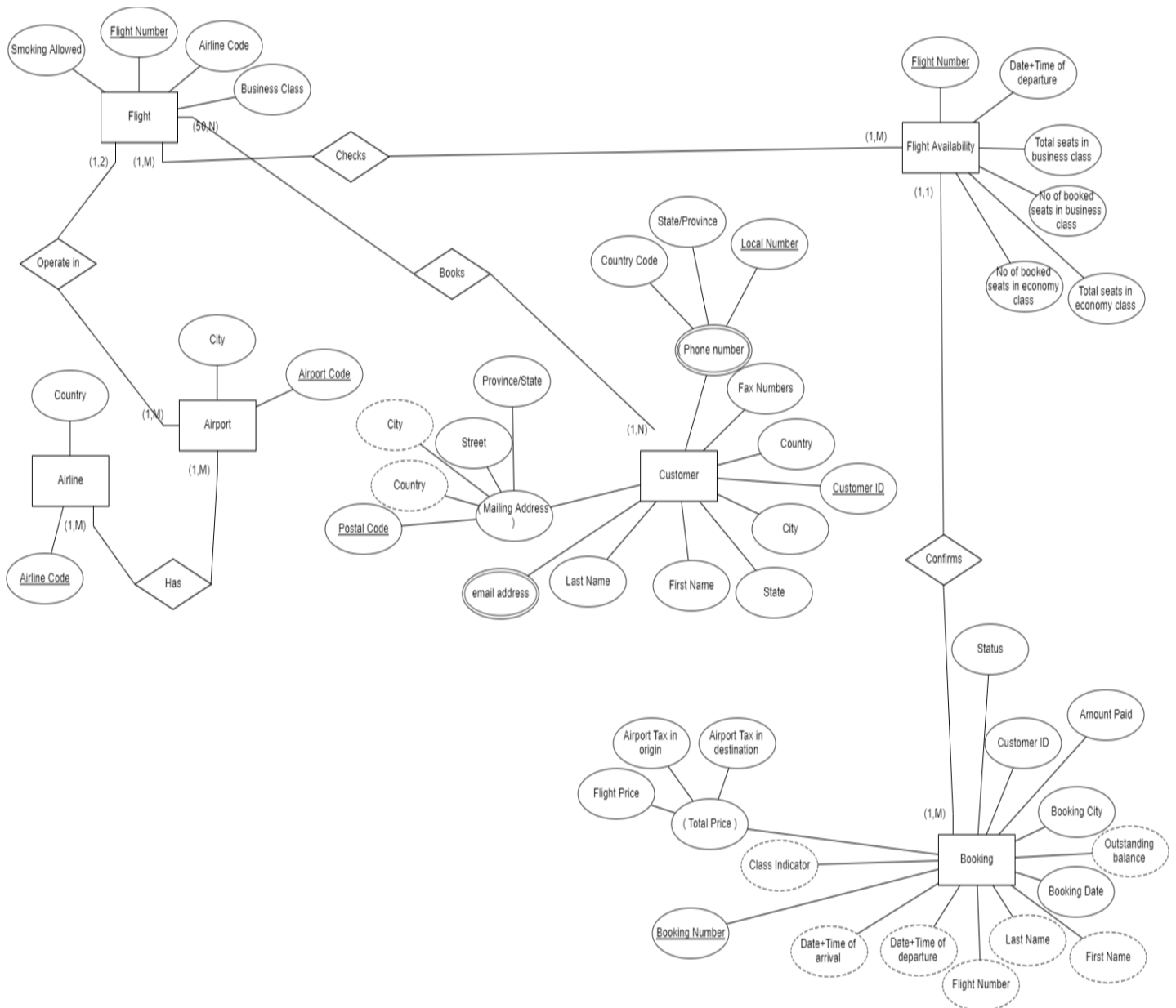
In each of the 12 cities, there is a (single) booking office. You are going to design a central air-reservation database to be used by all booking offices.

The flight has a unique flight number, airline code, business class indicator, smoking allowed indicator. Flight availability has flight number, date + time of departure, number of total seats available in business class, number of booked seats in business class, number of total seats available in economy class, and number of booked seats in economy class.

The customers may come from any country, not just the 6 above, and from any province/state, and from any city. Customer has first & last name, mailing address, zero or more phone numbers, zero or more fax numbers, and zero or more email addresses. Mailing address has street, city, province or state, postal code and country. Phone/fax number has country code, area code and local number. Email address has only one string, and no structure is assumed. A customer can book one or more flights. Two or more customers may have same mailing address and/or same phone number(s) and/or same fax number(s). But the email address is unique for each customer. First and last names do not have to be unique.

Booking has a unique booking number, booking city, booking date, flight number, date + time of departure (in local time, and time is always in hours and minutes), date + time of arrival (in local time), class indicator, total price (airport tax in origin + airport tax in destination + flight price – in local currency. The flightprice for business class is 1.5 times of the listed flight price), status indicator (three types: booked, Canceled – the customer canceled the booking, scratched – the

customer had not paid in full 30 days prior to the departure), customer who is responsible for payment, amount-paid-so far (in local currency), outstanding balance (in local currency), the first & last names to be printed on the ticket. The airport taxes must be stored in local currencies (i.e. Canadian dollars, US dollars, British Pounds, French francs, German marks, and Italian Liras). Since the exchange rates change daily, they also must be stored for calculations of all prices involved. Though France, Germany, and Italy have had a common currency for a while, we used the names of their original currencies to involve in this exercise currency exchange rates and their changes.



Question #3: A company called **EduTemps** provides temporary teaching staff to schools throughout England. The following unnormalized table lists information about the temporary teachers, the hours they worked, and the schools they worked at.

NIN	contractNo	hours	tName	schoolID	schoolName	schoolCity
2103	T9001	10	Brown A	S301	Park School	Manchester
2198	T9001	18	Khan R	S301	Park School	Manchester
2205	T9001	20	Lewis H	S114	Green School	Birmingham
2103	T9001	12	Brown A	S114	Green School	Birmingham

- The table shown above is susceptible to update anomalies. Provide examples of insertion, deletion, and update anomalies.
- Describe and illustrate the process of normalizing the table to 3NF, by identifying the functional dependencies represented by the attributes. State any assumptions you make.

Solution: (a)

Insertion

Anomaly:

- Suppose a new school is added to the system (e.g., “Riverdale School in Leeds”) but no teacher has been assigned to it yet. In this design, we cannot insert the new school because all records require a teacher assignment (due to attributes like NIN, tName, and hours being non-null).

Deletion Anomaly:

- If the only teacher (e.g., Brown A) assigned to “Green School” leaves the agency and their row is deleted, we also lose all data about “Green School” — including its schoolID, schoolName, and schoolCity.

Update Anomaly:

- If “Park School” changes its name to “Parkview School,” the name must be updated in every row where it appears. If one instance is missed, the database becomes inconsistent.

Solution (b):

Step 1: Identify Functional Dependencies

Assumptions:

- NIN uniquely identifies a teacher $\rightarrow$ determines tName
- schoolID uniquely identifies a school $\rightarrow$ determines schoolName and schoolCity
- The combination of NIN, schoolID, and contractNo determines hours

Functional Dependencies:

1. $NIN \rightarrow tName$
2. $schoolID \rightarrow schoolName, schoolCity$
3. $NIN, schoolID, contractNo \rightarrow hours$

Step 2: First Normal Form (1NF)

The table is already in 1NF because all values are atomic (no repeating groups or arrays).

Step 3: Second Normal Form (2NF)

To move to 2NF, we remove **partial dependencies**, i.e., attributes that depend on only part of the composite key (NIN, schoolID, contractNo).

Decompose into the following tables:

1. Teacher

NIN tName

2103 Brown A

2198 Khan R

2205 Lewis H

2. School

schoolID	schoolName
schoolCity S301	Park School
Manchester S114	Green
School Birmingham	

3. WorkRecord

NIN contractNo schoolID hours

2103 T9001 S301 10

2198 T9001 S301 18

2205 T9001 S114 20

2103 T9001 S114 12

Step 4: Third Normal Form (3NF)

We now check for **transitive dependencies** (i.e., non-key attributes depending on other non-key attributes). From above:

- $NIN \rightarrow tName$ (kept in separate table)
- $schoolID \rightarrow schoolName, schoolCity$ (kept in separate table)
- No transitive dependencies in

WorkRecord Thus, the schema is now in **3NF**.

Final 3NF Schema:

1. **Teacher(NIN, tName)**
2. **School(schoolID, schoolName, schoolCity)**
3. **WorkRecord(NIN, contractNo, schoolID, hours)**

Question#4:The casting office created the following un-normalized table, FILM_CASTING, to track casting details.

FILM_CASTING

FilmID, ActorID, FilmTitle, DirectorName, ActorName, AgentID, AgentPhone

Functional Dependencies (FDs) are given as:

1. $FilmID \rightarrow FilmTitle, DirectorName$
2. $ActorID \rightarrow ActorName, AgentID$
3. $AgentID \rightarrow AgentPhone$
4. $FilmID, ActorID \rightarrow$ All other attributes (No other single attribute determines the whole record).

- A. Identify the Primary Key (PK) for the FILM_CASTING table based on the given Functional Dependencies.
- B. The table is currently in 1NF. Which is the highest Normal Form (NF) it violates (2NF or 3NF)? Justify your answer by giving an example of a dependency violation.
- C. Decompose the table to Second Normal Form (2NF). List the new tables created and state the specific dependencies that were removed.
- D. Decompose the 2NF tables further to Third Normal Form (3NF). List all final tables in their 3NF form with their PK and FK
- E. In the original FILM_CASTING table, AgentPhone is stored repeatedly for every film an actor is in. Which type of data redundancy anomaly (Insert, Update, or Delete) would this cause if the agent's phone number changes?

Solution:

A.

Primary Key (PK): The only key that determines all attributes is the composite key: FilmID, ActorID.

B.

The table violates Second Normal Form (2NF).

Violation: There are Partial Dependencies where non-key attributes depend on only part of the composite PK (FilmID or ActorID):

* FilmID→FilmTitle, DirectorName

* ActorID→ActorName, AgentID

C.

Decomposition to Second Normal Form (2NF):

We separate the attributes that are partially dependent on each key part.

Tables in 2NF:

1. FILM_T (FilmID, FilmTitle, DirectorName)

2. ACTOR_T (ActorID, ActorName, AgentID, AgentPhone)

3. CASTING_2NF (FilmIDFK, ActorIDFK)

D.

Decomposition to Third Normal Form (3NF):

The ACTOR_T table still violates 3NF due to the Transitive Dependency: ActorID→AgentID and AgentID→AgentPhone (Non-Key → Non-Key). We must remove this.

Final 3NF Relations:

1. FILM (FilmID, FilmTitle, DirectorName)

2. ACTOR (ActorID, ActorName, AgentIDFK)

3. AGENT (AgentID, AgentPhone)

4. CASTING (FilmIDFK, ActorIDFK)

E.

This would cause an Update Anomaly.

If the agent's phone number changes, the studio would have to update every single row in the FILM_CASTING table where that actor appears. Failing to update all rows would lead to data inconsistency.

Question#5: Examine the **City College Enrollment Form** shown below:

City College ENROLLMENT FORM				
Student ID: <u>S1001</u>		First Name: <u>James</u>		
Last Name: <u>Smith</u>		Address: <u>123 Oak Street</u>		
Course ID	Course Name	Credits	Instructor Name	Enrollment Date
CS101	Introduction to Computer Science	3	Dr. Baker	15/01/2024
MA202	Calculus II	4	Dr. Johnson	15/01/2024
PH301	Physics	3	Dr. Lee	15/01/2024

1. Identify the **functional dependencies** represented by the attributes shown in the form above. State any assumptions that you make about the data and the attributes.
2. Describe and illustrate the process of **normalizing** the attributes shown in the form above to produce a set of well-designed **3NF (Third Normal Form)** relations.
3. Identify the **primary**, **alternate**, and **foreign keys** in your 3NF relations.

Solution

a.

Assumptions:

1. Each StudentID uniquely identifies one student.
2. Each CourseID uniquely identifies one course.
3. Each course is taught by exactly one instructor.
4. A student may enroll in multiple courses.
5. EnrollmentDate refers to the date the student enrolled in that course.

Functional Dependencies:

1. StudentID $\rightarrow$ FirstName, LastName, Address (A student's details depend on StudentID)
2. CourseID $\rightarrow$ CourseName, Credits, InstructorName (A course's details depend on CourseID)
3. (StudentID, CourseID) $\rightarrow$ EnrollmentDate
(A student's enrollment in a course determines the date of enrollment)

b.

Normalization Process

Unnormalized Relation (UNF)

ENROLLMENT_FORM(StudentID, FirstName, LastName, Address, CourseID, CourseName, Credits, InstructorName, EnrollmentDate)

This table has **repeated data** (e.g., student info repeated for each course, and course info repeated for each student).

First Normal Form (1NF)

All attributes are atomic (no multi-valued fields).

→ Already in 1NF.

Second Normal Form (2NF)

Remove partial dependencies (attributes depending only on part of a composite key).

- StudentID determines student details → move them to a separate **Student** table.
- CourseID determines course details → move them to a separate **Course** table.
- Keep (StudentID, CourseID, EnrollmentDate) in the **Enrollment** table.

Third Normal Form (3NF)

Remove transitive dependencies (non-key attributes depending on other non-key attributes).

- InstructorName depends on CourseID, not directly on Enrollment → so create an **Instructor** table.
- Link Course table to Instructor by InstructorID.

Final 3NF Relations

1. **STUDENT**
2. Student(StudentID, FirstName, LastName, Address)
 - PK: StudentID
3. **INSTRUCTOR**
4. Instructor(InstructorID, InstructorName)
 - PK: InstructorID
5. **COURSE**
6. Course(CourseID, CourseName, Credits, InstructorID)
 - PK: CourseID
 - FK: InstructorID → Instructor(InstructorID)
7. **ENROLLMENT**
8. Enrollment(StudentID, CourseID, EnrollmentDate)
 - PK: (StudentID, CourseID)
 - FK1: StudentID → Student(StudentID)
 - FK2: CourseID → Course(CourseID)

c.

Table	Primary Key	Alternate Key(s)	Foreign Key(s)
Student	StudentID	—	—
Instructor	InstructorID	—	—
Course	CourseID	—	InstructorID
Enrollment	(StudentID, CourseID)	—	StudentID, CourseID

Good Luck!